1. Introduction and Option Selected

For the CS-327 Computer Communication Networks Complex Engineering Problem (Fall 2026), our group selected **Option B: Mini DNS Service** and engineered **SentraDNS** (`MYDNS/1.0`). Rather than consuming existing application protocols as black-box libraries, SentraDNS specifies an original binary-framed name resolution protocol implemented directly over raw UDP datagram sockets (`socket.SOCK_DGRAM`) across three independent network hosts.

Option B was selected because its three-tier architecture maps cleanly onto three physical/virtual endpoints and yields a rigorous empirical contrast between uncached four-hop resolution (**Cache Miss**) and cached two-hop resolution (**Cache Hit**), directly validating the nodal delay decomposition model and quantifying bandwidth and energy savings at scale:

- **Host 1 — DNS Client (`src/client.py`):** Generates `TYPE_A` lookup queries, executes application-layer Stop-and-Wait ARQ retransmission, validates 16-bit Transaction IDs, and measures round-trip time (RTT) via `time.perf_counter()`.
- **Host 2 — Caching Resolver (`src/resolver.py`):** Concurrently processes incoming datagrams via a 32-worker `ThreadPoolExecutor`, maintains a thread-safe Time-To-Live (TTL) cache (`DNSCache`), and forwards cache misses upstream.
- **Host 3 — Authoritative Name Server (`src/authoritative_server.py`):** Serves master zone records from `src/zone.json`, enforces strict wire-length framing, and answers queries with IPv4 mappings or `NXDOMAIN`.

2. Protocol Specification (`MYDNS/1.0`)

`MYDNS/1.0` is a stateless, request–response binary protocol encoded in Network Byte Order (Big-Endian). Any third-party group can implement a compatible client, resolver, or authoritative server solely from the specification below.

2.1 Message Types, Query Types, and Status Codes

2.2 Binary Wire Framing (`!HBBBxH`)

Every datagram starts with a fixed 8-byte (64-bit) binary header packed via Python's `struct` format string `!HBBBxH`, followed by a 1-byte length prefix $L \in [1, 255]$, an L-byte ASCII domain name, and a conditional 4-byte IPv4 trailer (`MSG_RESPONSE` only):

- **Bytes 0–1 (`tx_id`, `!H`):** Random 16-bit unsigned Transaction ID (`0x0001`–`0xFFFF`) echoed verbatim in replies.
- **Byte 2 (`msg_type`, `!B`):** Message role (`0x01`, `0x02`, or `0x03`).
- **Byte 3 (`query_type`, `!B`):** Record type (`0x01` = `TYPE_A`).
- **Byte 4 (`status`, `!B`):** Outcome status code (`0x00`–`0x03`); set to `0x00` in queries.
- **Byte 5 (`reserved`, `x`):** Zero-padded alignment byte (`0x00`) aligning the 16-bit `ttl` on an even boundary.
- **Bytes 6–7 (`ttl`, `!H`):** Remaining cache lifetime in seconds (`0`–`65535`); `0` in queries and errors.
- **Byte 8 (`domain_len`, `!B`):** Exact byte length L of the ASCII domain string ($1 \leq L \leq 255$).
- **Bytes $9 \ldots 8 + L$ (`domain`):** Unterminated ASCII domain string (e.g., `portal.neduet.local`, $L = 19$).

Header Field	Constant Name	Hex	Protocol Semantics
msg_type	MSG_QUERY	0x01	Client/resolver lookup request; omits 4-byte IPv4 trailer.
	MSG_RESPONSE	0x02	Resolver/authoritative reply; includes 4-byte IPv4 trailer.
	MSG_ERROR	0x03	Wire-level syntax/framing violation; omits IPv4 trailer.
query_type	TYPE_A	0x01	Standard 32-bit IPv4 host address record query.
status	STATUS_SUCCESS	0x00	Domain resolved; valid IPv4 address and positive TTL included.
	STATUS_NXDOMAIN	0x01	Domain absent from authoritative zone (0.0.0.0, TTL=0).
	STATUS_SERVER_FAILURE	0x02	Resolver timed out awaiting Host 3 (0.0.0.0, TTL=0).
	STATUS_FORMAT_ERROR	0x03	Truncated header, invalid opcode, or trailing garbage bytes.

Table 1. MYDNS/1.0 Message Opcodes and Status Codes

- **Bytes $9+L \ldots 12+L$ (ip_address):** 4-byte network-order IPv4 address via socket.inet_aton() (MSG_RESPONSE only).

  Packet length is deterministic from the header fields alone:

$$\text{QUERY / ERROR packet length} = 9 + L \text{ bytes} \tag{1}$$

$$\text{RESPONSE packet length} = 13 + L \text{ bytes} \tag{2}$$

For portal.neduet.local ($L = 19$): QUERY $= 28$ B, RESPONSE $= 32$ B. For MSG_ERROR with placeholder domain bad.request ($L = 11$): ERROR $= 20$ B.

2.3 Legal Exchange Sequences and Receiver State Actions

1. **Cache Hit (2 Hops):** Host 1 sends MSG_QUERY ($9 + L$ bytes) to Host 2. Host 2 validates exact framing, locates a non-expired entry in DNSCache, computes remaining TTL $\lceil t_{\text{exp}} - t_{\text{now}} \rceil$, and transmits MSG_RESPONSE (STATUS_SUCCESS, $13 + L$ bytes) directly to Host 1.
2. **Cache Miss (4 Hops):** On cache miss or expiry, Host 2 forwards the raw query bytes over a connected UDP socket to Host 3. Host 3 performs a case-insensitive lookup in zone.json while preserving original domain casing in its reply, returning STATUS_SUCCESS or STATUS_NXDOMAIN. Host 2 caches positive responses (status $=$ 0x00 and TTL $>$ 0), relays negative responses without caching, and forwards the reply to Host 1.
3. **Non-Existent Domain (nxdomain):** Host 3 returns MSG_RESPONSE / STATUS_NXDOMAIN with ip_address = 0.0.0.0 and TTL = 0. Host 2 relays this to Host 1 unchanged and does *not* cache the result (TTL = 0 prevents storage).
4. **Authoritative Server Unreachable (server_failure):** If Host 3 is offline, Host 2's single 1.0 s upstream attempt times out. Host 2 immediately returns MSG_RESPONSE / STATUS_SERVER_FAILURE (0.0.0.0, TTL=0) to Host 1 at approximately $T = 1.0$ s — safely before Host 1's 1.5 s Stop-and-Wait retransmission timer fires.

5. **Malformed Packet Handling:** Any datagram < 10 B, containing invalid opcodes, trailing bytes beyond $9 + L$ (or $13 + L$), or a non-`MSG_QUERY` sent by a client is rejected at Host 2 (never forwarded to Host 3) with `MSG_ERROR` (`STATUS_FORMAT_ERROR`, `ttl=0`, placeholder domain `bad.request`, 20 B), copying the first 2 bytes as `tx_id`. Host 3 applies the identical validation to direct traffic and ignores stray `MSG_RESPONSE`/`MSG_ERROR` packets.

2.4 Client Protocol State Machine

Figure 1 captures the complete Stop-and-Wait ARQ state machine executed by `src/client.py` for a single query lifecycle.

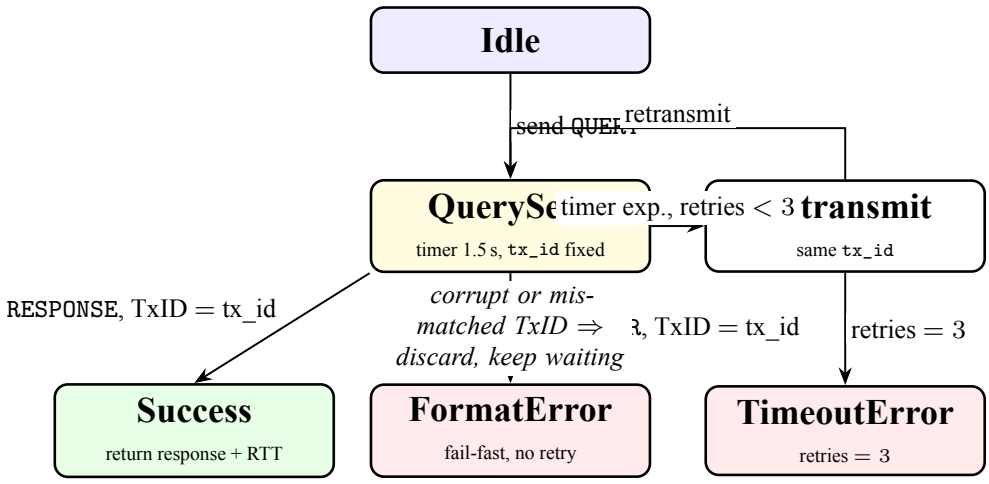


Figure 1. Client Stop-and-Wait ARQ State Machine (`src/client.py`)

3. System and Deployment Architecture

SentraDNS is deployed across three independent network endpoints on a shared IPv4 subnet so every link traversal crosses real network interfaces rather than loopback (`127.0.0.1`). Figure 2 illustrates the three-tier topology across Links 1 and 2, while Table 2 and Figure 3 detail the physical host interface configurations on the shared `26.0.0.0/8` subnet.

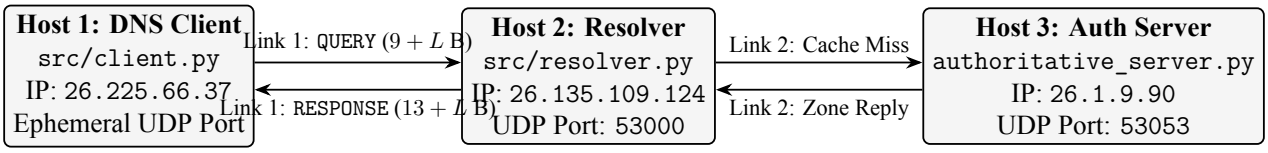
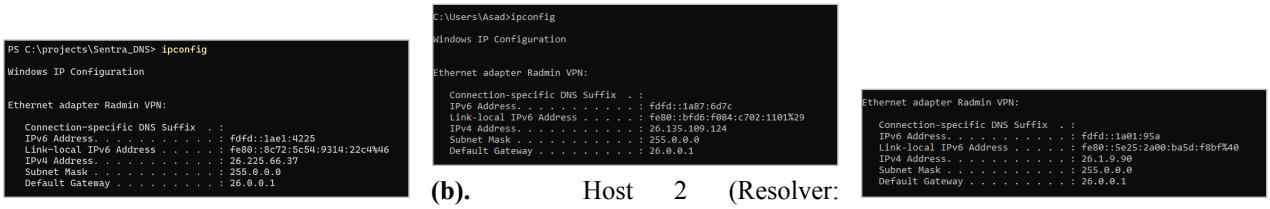


Figure 2. Three-Host SentraDNS Network Deployment Topology

Host	OS / Machine	IPv4 Address	UDP Port	Interface Capture (`ipconfig` / `ip addr`)
Host 1	Windows 11	`26.225.66.37`	Ephemeral (63249)	Radmin VPN (`255.0.0.0`, MAC: `02:50:FA:D8:EA:6A`)
Host 2	Windows 11	`26.135.109.124`	`53000`	Radmin VPN (`255.0.0.0`, MAC: `02:50:3D:5C:C0:2D`)
Host 3	Windows 11	`26.1.9.90`	`53053`	Radmin VPN (`255.0.0.0`, Gateway: `26.0.0.1`)

Table 2. IP Addressing Plan and Host Interface Configuration



(a). Host 1 (Client: `26.225.66.37`) **(b).** Host 2 (Resolver: `26.135.109.124`) **(c).** Host 3 (Auth: `26.1.9.90`)

Figure 3. Host Network Interface Configurations on Shared Subnet (`26.0.0.0/8`)

4. Implementation Notes and Transport Layer Justification

All source modules (`protocol.py`, `client.py`, `resolver.py`, `authoritative_server.py`) rely strictly on Python's native `socket`, `struct`, `threading`, `concurrent.futures`, and `json` libraries. `src/protocol.py` acts as the sole shared dependency so all three nodes share an identical serialization contract.

4.1 Transport Layer Choice: UDP vs. TCP (R4)

Every `MYDNS/1.0` message type (`MSG_QUERY`, `MSG_RESPONSE`, `MSG_ERROR`) is carried over **UDP** rather than TCP, justified against the four application service requirements:

1. **Timing (Latency):** DNS resolution is a small, atomic request–response lookup (28 B query, 32 B response) executed before application connections begin. Choosing TCP would impose a mandatory 3-way handshake (1 RTT setup) plus connection teardown (`FIN`/`ACK`), adding 1–2 extra RTTs to every uncached lookup.
2. **Reliable Data Transfer (RDT):** Because DNS queries are idempotent, full TCP stream ordering and congestion control are unnecessary. Instead, `client.py` implements application-layer **Stop-and-Wait ARQ** (1.5 s per-attempt deadline, up to 3 attempts, matched by 16-bit `tx_id` while discarding corrupt/stale background packets), and `resolver.py` performs a single 1.0 s upstream attempt.
3. **Throughput:** High-volume DNS traffic consists of thousands of independent micro-datagrams. UDP uses an 8 B header instead of TCP's 20 B minimum header and avoids per-client socket descriptor exhaustion on Host 2's 32-thread worker pool.
4. **Security:** Within a controlled campus LAN/hotspot deployment, plaintext lookup latency is prioritized while `tx_id` validation and exact payload length checks ($9 + L$ / $13 + L$) reject malformed or trailing-byte injection packets.

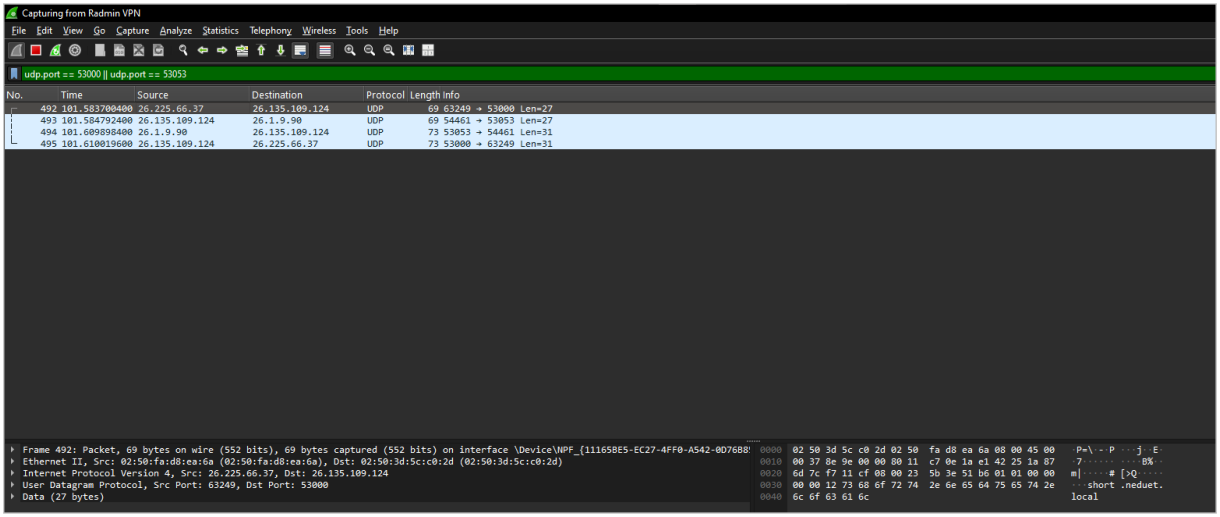
5. Packet Capture and Encapsulation Analysis

Wire traffic across our three-node network deployment was captured in Wireshark filtered on `udp.port == 53000 || udp.port == 53053`. A query for `short.neduet.local` ($L = 18$) produces a 69-byte Ethernet frame demonstrating complete layered encapsulation:

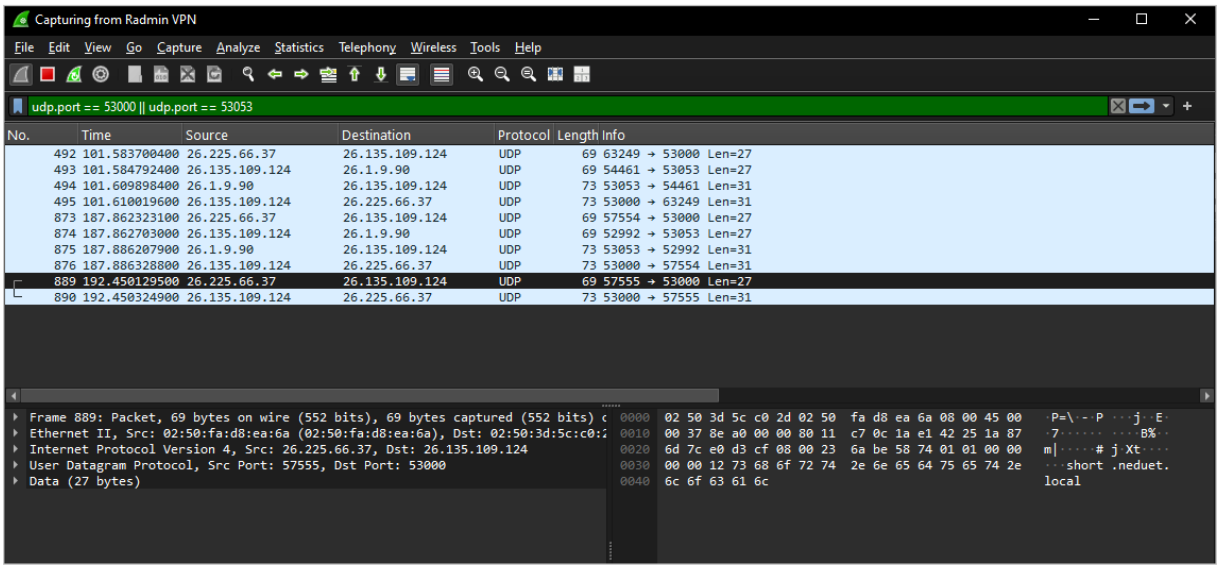
- **Link Layer (Ethernet II — 14 bytes):** Destination MAC (`02:50:3d:5c:c0:2d`, Host 2), Source MAC (`02:50:fa:d8:ea:6a`, Host 1), and EtherType `0x0800` (IPv4).
- **Network Layer (IPv4 — 20 bytes):** Version 4, IHL 5, Protocol 17 (UDP), Total Length 55 (20 B IP+8 B UDP+27 B MYDNS), Source IP `26.225.66.37`, Destination IP `26.135.109.124`.
- **Transport Layer (UDP — 8 bytes):** Source Port 63249 (ephemeral client port), Destination Port 53000, Length 35 (8 B UDP + 27 B payload), UDP Checksum.

- **Application Layer (`MYDNS/1.0` — 27 bytes):** Bytes `0--1`: `tx_id` (`0x4031`); Byte `2`: `01` (`MSG_QUERY`); Byte `3`: `01` (`TYPE_A`); Byte `4`: `00` (`SUCCESS`); Byte `5`: `00` (reserved pad); Bytes `6--7`: `00 00` (`TTL=0`); Byte `8`: `12` ($L = 18$); Bytes `9--26`: ASCII `short.neduet.local`.

Figure 4 documents the side-by-side empirical verification in Wireshark: subfigure (a) demonstrates the complete 4-packet exchange across Links 1 and 2 during an uncached resolution, while subfigure (b) confirms that Link 2 is completely bypassed during a cache hit.



(a). Uncached 4-Hop Resolution (**Cache Miss**): Wireshark Frames 492–495 (Client ↔ Resolver ↔ Auth Server)



(b). Cached 2-Hop Resolution (**Cache Hit**): Wireshark Frames 889–890 (Link 2 Completely Bypassed)

Figure 4. Annotated Wireshark Packet Captures for `short.neduet.local` ($L = 18$, Port 53000/53053)

6. Performance Measurement and Results

Using `tests/benchmark_runner.py`, we evaluated SentraDNS over $N = 10$ repeated trials ($N \geq 5$) comparing **Cache Miss** (querying uncached `bench01`–`bench10.neduet.local` across Links 1 and 2) against **Cache Hit** (querying cached `portal.neduet.local` across Link 1 only). Sample spread is computed using Bessel's corrected standard deviation $\sigma = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \mu)^2}$, and sustained throughput is calculated as $\text{QPS} = N / \sum t_i$ and $\text{KB/s} = (\text{QPS} \times B_{\text{client}})/1024$. The measured metrics are summarized in Table 3, and an isolated single-query comparison is illustrated in Figure 5.

Performance Metric	Cache Miss (4 Hops)	Cache Hit (2 Hops)
Sample Trials (N)	10	10
Mean RTT (μ)	27.99 ms	1.85 ms
Sample Std Deviation (σ)	$\pm$0.87 ms	$\pm$0.37 ms
Min–Max Spread	26.57–29.29 ms	1.46–2.59 ms
Application Throughput (QPS)	35.7 QPS	540.5 QPS
Payload Throughput (KB/s)	2.16 KB/s	31.67 KB/s
Total Wire Application Payload / Query	124 B ($2 \times [29 + 33$ B for L=20])	60 B ($28 + 32$ B for L=19)

Table 3. Multi-Host Empirical Performance Comparison: Cache Miss vs. Cache Hit (15.13× Latency Speedup, 15.14× Throughput Gain)

```
MYDNS/1.0 Resolution Result
========================================================================
Queried Domain:   short.neduet.local
Resolver Host:    26.135.109.124:53000
Transaction ID:   0x4031 (16433)
Response Status:  SUCCESS (0x00)
Round-Trip Time:  123.71 ms
Resolved IPv4:    10.0.3.1
Time-To-Live:     10 seconds
Path Performance: CACHE MISS (Full Upstream Traversal, RTT >= 10.0ms)
------------------------------------------------------------------------
[SUCCESS] short.neduet.local -> 10.0.3.1 (TTL: 10s, RTT: 123.71ms)
========================================================================
PS C:\projects\Sentra_DNS> python src/client.py --resolver-ip 26.135.109.124 --domain short.neduet.local
[CLIENT] Sending MYDNS/1.0 query for 'short.neduet.local' to 26.135.109.124:53000...
========================================================================
MYDNS/1.0 Resolution Result
========================================================================
Queried Domain:   short.neduet.local
Resolver Host:    26.135.109.124:53000
Transaction ID:   0x5874 (22644)
Response Status:  SUCCESS (0x00)
Round-Trip Time:  2.55 ms
Resolved IPv4:    10.0.3.1
Time-To-Live:     6 seconds
Path Performance: CACHE HIT (Fast Path, RTT < 10.0ms)
------------------------------------------------------------------------
[SUCCESS] short.neduet.local -> 10.0.3.1 (TTL: 6s, RTT: 2.55ms)
========================================================================
PS C:\projects\Sentra_DNS>
```

Figure 5. Empirical Client Demonstration of Cache Miss (123.71 ms, TTL $= 10$ s) vs. Cache Hit (2.55 ms, TTL $= 6$ s, 48.5× Speedup)

6.1 Nodal Delay Model Interpretation

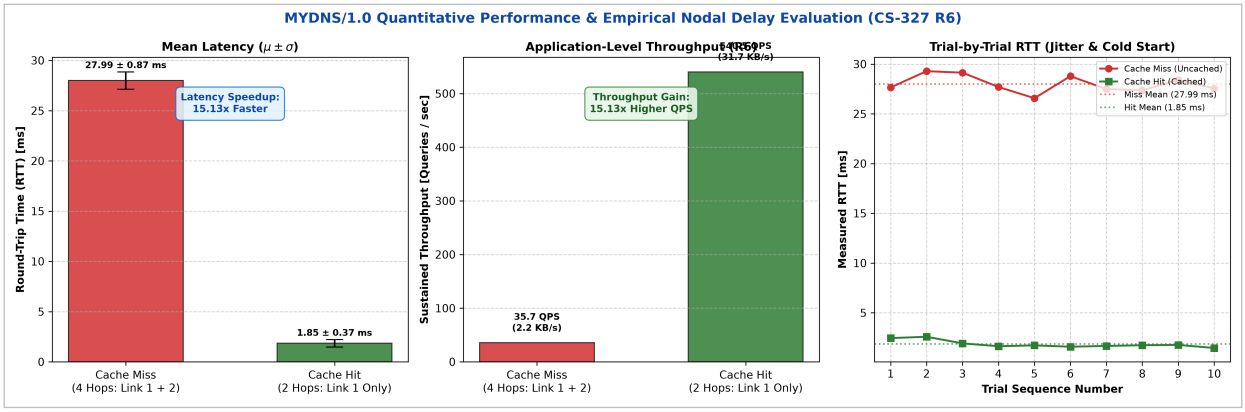
In the nodal delay equation $d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$, a Cache Miss traverses four network hops (Host $1 \to 2 \to 3 \to 2 \to 1$), incurring four transmission delays (L/R), four propagation delays (d/s), two intermediate queueing intervals, plus zone lookup and cache write processing (d_{proc}). A Cache Hit eliminates Link 2 entirely, cutting d_{trans} and d_{prop} in half and resolving from Host 2's in-memory `DNSCache` in microseconds. Across our 10-trial benchmark, this reduced mean resolution delay from 27.99 ms to 1.85 ms—delivering an empirical 15.13× **latency reduction** and boosting application throughput from 35.7 QPS to 540.5 QPS (15.14× gain).

Figure 6 synthesizes our complete benchmark findings: subfigure (a) presents the 3-panel publication chart, while subfigures (b) and (c) display the corresponding live benchmark console output and the Wireshark packet capture showing the stark reduction in packet volume.

7. Testing

7.1 Test Suite Overview

The `tests/` directory contains 6 automated Python `unittest` files executed with `python -m pytest tests/ -v` (or `python -m unittest discover`), collectively covering 72 unit and integration



(a). Multi-Trial Empirical Benchmark: Mean RTT ($\mu \pm \sigma$), Sustained Throughput (QPS & KB/s), and Jitter Progression

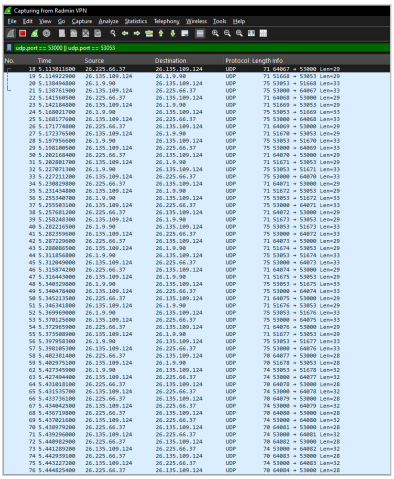
```
[PHASE 1] Executing 10 Trials under CACHE MISS Condition...
Each trial queries an uncached benchmark domain to force full 4-hop resolution.
  Trial  1/10: RTT =  27.65 ms | Domain = bench01.neduet.local      | Status = SUCCESS
  Trial  2/10: RTT =  29.29 ms | Domain = bench02.neduet.local      | Status = SUCCESS
  Trial  3/10: RTT =  29.14 ms | Domain = bench03.neduet.local      | Status = SUCCESS
  Trial  4/10: RTT =  27.70 ms | Domain = bench04.neduet.local      | Status = SUCCESS
  Trial  5/10: RTT =  26.57 ms | Domain = bench05.neduet.local      | Status = SUCCESS
  Trial  6/10: RTT =  28.78 ms | Domain = bench06.neduet.local      | Status = SUCCESS
  Trial  7/10: RTT =  27.49 ms | Domain = bench07.neduet.local      | Status = SUCCESS
  Trial  8/10: RTT =  27.37 ms | Domain = bench08.neduet.local      | Status = SUCCESS
  Trial  9/10: RTT =  28.38 ms | Domain = bench09.neduet.local      | Status = SUCCESS
  Trial 10/10: RTT =  27.55 ms | Domain = bench10.neduet.local      | Status = SUCCESS

[PHASE 2] Priming Cache with 'portal.neduet.local'...
  Cache primed in 27.52 ms (Status: SUCCESS)

[PHASE 3] Executing 10 Consecutive Trials under CACHE HIT Condition...
Each trial queries the cached domain to test the fast path (Link 1 only).
  Trial  1/10: RTT =   2.45 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial  2/10: RTT =   2.59 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial  3/10: RTT =   1.92 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial  4/10: RTT =   1.63 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial  5/10: RTT =   1.72 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial  6/10: RTT =   1.58 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial  7/10: RTT =   1.66 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial  8/10: RTT =   1.73 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial  9/10: RTT =   1.77 ms | Domain = portal.neduet.local      | Remaining TTL = 120s
  Trial 10/10: RTT =   1.46 ms | Domain = portal.neduet.local      | Remaining TTL = 120s

================================================================================
STATISTICAL BENCHMARK COMPARISON TABLE (CS-327 Rubric Criterion 3 & R6)
================================================================================
| Metric                     | Cache Miss (4 Hops)   | Cache Hit (2 Hops)    |
|:---------------------------|:----------------------|:----------------------:|
| Sample Trials (N)          |                    10 |                    10 |
| Mean RTT (ms)              |              27.99 ms |               1.85 ms |
| Std Deviation (sigma)      |               0.87 ms |               0.37 ms |
| Minimum Latency            |              26.57 ms |               1.46 ms |
| Maximum Latency            |              29.29 ms |               2.59 ms |
| Throughput (Queries/sec)   |               35.7 QPS |             540.5 QPS |
| Throughput (Payload KB/s)  |              2.16 KB/s |             31.67 KB/s |
| Client Payload / Query     |               62 bytes |               60 bytes |
| Total Wire Traffic / Query |              124 bytes |               60 bytes |
| Upstream Link 2 Traversal  | Host 2 <-> Host 3 (Yes) |  Bypassed (Zero Load) |
--------------------------------------------------------------------------------
Empirical Latency Speedup:     15.13x faster on Cache Hit
```

(b). Benchmark Terminal Output ($N = 10$ Trials)



(c). Benchmark Wireshark Trace (4-Pkt vs. 2-Pkt)

Figure 6. Comprehensive Multi-Host Performance Benchmark and Verification ($N = 10$ Trials)

tests across protocol serialization, client ARQ behavior, resolver cache logic, end-to-end datagram handling, and benchmark statistics. Table 4 summarizes the test distribution across modules.

7.2 Key Test Cases

Table 5 documents the most representative test cases, their expected results, and observed outcomes from automated execution.

In addition, `tests/auth_local_check.py` runs 8 live integration checks directly against Host 3, including a **50-query concurrent stress test** using `ThreadPoolExecutor(max_workers=50)` across all 10 benchmark domains, verifying zero dropped or mismatched responses under simultaneous load. Figure 7 provides the terminal confirmation of all 72 automated test cases passing.

8. Design Tensions and Engineering Justification

1. **Reliability vs. Latency (UDP + Coordinated Stop-and-Wait ARQ):** TCP guarantees reliability but penalizes every single-packet DNS query with a 3-way handshake; raw UDP avoids handshake latency but drops lost packets silently. We chose UDP paired with a 1.5 s client Stop-and-Wait timer and a 1.0 s resolver upstream timeout. Under normal conditions queries finish in 1 RTT; under packet loss, Host 1 retransmits cleanly, whereas if Host 3 is down, Host 2 reports `SERVER_FAILURE` at 1.0 s before Host 1's 1.5 s retry fires.
2. **Caching Freshness vs. Traffic Reduction (Dynamic TTL Decrementing):** Static caching eliminates upstream traffic indefinitely but risks serving stale IP addresses; disabling caching

File	Tests	What It Covers	Socket Used
`test_protocol.py`	17	Binary SerDes, big-endian layout, domain bounds ($L = 1$–255), all error types, round-trip fidelity	None (in-memory)
`test_client.py`	8	Stop-and-Wait ARQ, retransmit, timeout, TxID filtering, NXDOMAIN, SERVER_FAILURE, fast-fail on MSG_ERROR	Real OS UDP (`127.0.0.1:0`)
`test_resolver.py`	10	`DNSCache` TTL countdown, expiry, case-insensitivity, purge, overwrite	None (fake clock)
`test_resolver_server.py`	14	`handle_datagram()` unit tests + end-to-end stack with real sockets	Real OS UDP
`test_resolver_validation.py`	19	`validate_client_datagram()`: trailing bytes, bad opcodes, non-QUERY types	None (in-memory)
`test_benchmark.py`	4	Statistical μ/σ/QPS formulas, Matplotlib chart generation	Real OS UDP

Table 4. Automated Test Suite Summary (72 Tests Total, 100% Pass Rate)

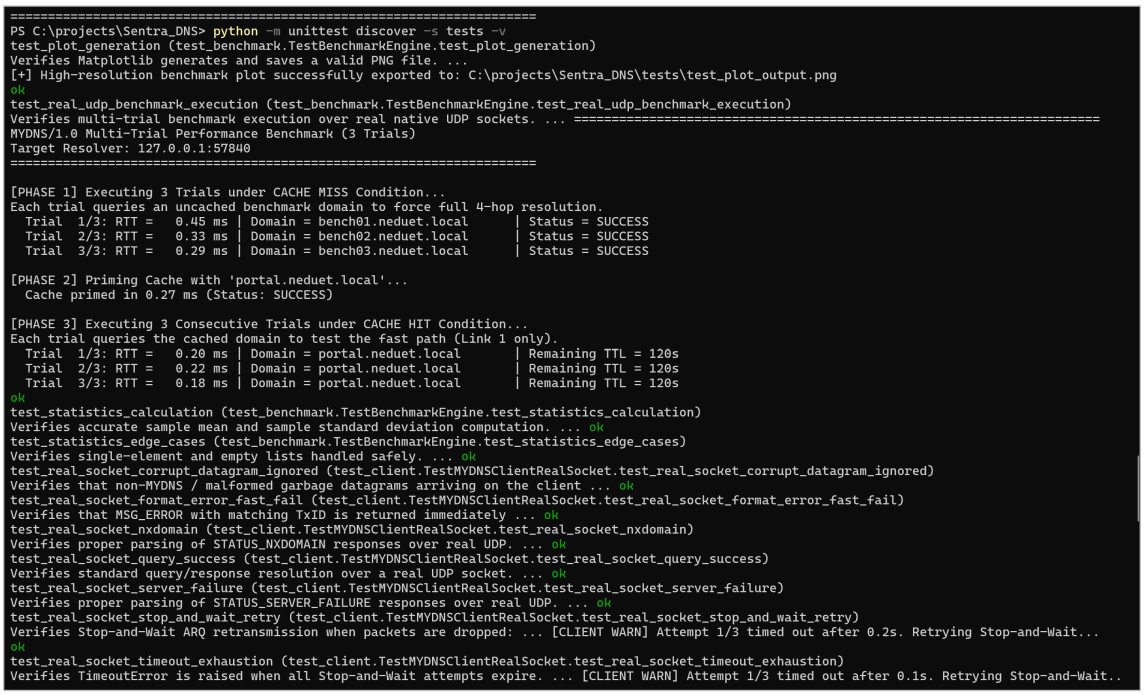


Figure 7. Automated Test Suite Terminal Output (`python -m unittest discover -s tests -v`, 72 Tests Passing)

#	Test Name	Scenario	Expected Outcome
1	`test_query_pack_unpack`	Query serialization (L=19)	28 B wire, `ip_address=None`, round-trip equal
2	`test_response_success_pack_unpack`	Response serialization	32 B wire, `ip=10.0.1.5`, `ttl=120`
3	`test_big_endian_wire_layout`	Big-endian byte alignment	`wire[0:2]=0xA1B2`, `wire[2]=0x01`, `wire[6:8]=0x0078`
4	`test_minimum_domain_length`	Boundary L=1	Query 10 B, Response 14 B
5	`test_maximum_domain_length`	Boundary L=255	Query 264 B, Response 268 B
6	`test_zero_length_domain_rejected`	L=0 rejected at pack & unpack	`ValueError` raised
7	`test_real_socket_stop_and_wait_retry`	Server drops attempt 1, replies on attempt 2	2 wire datagrams sent; SUCCESS returned
8	`test_real_socket_timeout_exhaustion`	All 3 attempts dropped	`TimeoutError`; exactly 3 datagrams sent
9	`test_real_socket_tx_id_filtering`	Wrong TxID sent first, then correct	Wrong TxID ignored; correct `ip=10.0.1.5` accepted
10	`test_real_socket_format_error_fast_fail`	Server returns MSG_ERROR on first attempt	Only 1 attempt; `MSG_ERROR` returned immediately (no retry)
11	`test_hit_ttl_counts_down`	Cache advanced 30 s after insertion (TTL=120)	`remaining_ttl=90`; dynamic countdown confirmed
12	`test_nxdomain_relayed_not_cached`	Two queries for absent domain	2 upstream

guarantees freshness at $2\times$ traffic cost. `DNSCache` stores `expires_at = clock() + ttl` and returns $\lceil \text{expires_at} - \text{now} \rceil$ on every hit, lazily evicting expired records (verified with `short.neduet.local`, TTL $= 10$ s).

3. **Compactness vs. Readability (Hybrid Binary Header + ASCII Domain):** Pure JSON payloads add > 100 B of overhead and variable parsing latency, whereas RFC 1035 label-pointer compression complicates parser logic. `MYDNS/1.0` uses a fixed 8-byte binary header (`!HBBBxH`) for $O(1)$ `struct.unpack` parsing while keeping the domain string in plain ASCII for human-readable Wireshark inspection.

9. Sustainability and Efficiency Analysis

For `portal.neduet.local` ($L = 19$), an uncached resolution transmits $28\,\text{B} + 28\,\text{B} + 32\,\text{B} + 32\,\text{B} =$ **120** bytes of application payload (288 bytes including 42 B Ethernet/IP/UDP headers per datagram). A cache hit requires only $28\,\text{B} + 32\,\text{B} =$ **60** bytes of application payload (144 bytes on the wire)—a **50.0% reduction** in transferred bytes and a **100% elimination** of Host 3 socket interrupts and zone lookups.

Projecting to a campus population of $10,000$ users issuing 50 queries/hour ($500,000$ queries/hour) at an 85% cache hit ratio ($425,000$ hits/hour):

$$\Delta\text{Bytes}_{\text{wire}} = 425,000 \times (288 - 144)\,\text{B/hr} = 61.2\,\text{MB/hr} \approx 1.47\,\text{GB/day} \approx 536.3\,\text{GB/year}.$$

Furthermore, omitting the 4-byte IPv4 field from queries saves 4 B/query (17.5 GB/year at this query rate), while compact binary framing cuts payload volume by $\sim$70% compared to JSON-over-HTTP, directly lowering NIC radio/switch switching energy and CPU wakeups at scale.

10. Behaviour Under Failure

SentraDNS was verified against automated unit/integration tests and live multi-host fault injection across three abnormal conditions:

- **Authoritative Server Unreachable (Host 3 Stopped):** Host 2's single 1.0 s upstream attempt expires prior to Host 1's 1.5 s retry timer, returning `MSG_RESPONSE` with `STATUS_SERVER_FAILURE` (`0x02`) and preventing duplicate client retransmissions.
- **Transient UDP Datagram Loss:** Dropping Attempt 1 triggers Host 1's Stop-and-Wait timer (`[CLIENT WARN] Attempt 1/3 timed out... Retrying Stop-and-Wait...`), recovering on Attempt 2 with the same `tx_id`.
- **Malformed / Trailing-Byte Packets:** Sending truncated bytes (e.g., `0xDEADBE`) or valid queries with trailing garbage triggers an immediate `MSG_ERROR` (`STATUS_FORMAT_ERROR`, `0x03`) from Host 2 without forwarding upstream, and `client.py` fails fast on matching `tx_id` without retrying.

Figure 8 documents the empirical fault injection and negative resolution demonstrations: subfigure (a) shows Host 1 detecting server failure when Host 3 is down; subfigure (b) shows Host 2 logging the upstream timeout; and subfigure (c) shows Host 1 receiving an authoritative `STATUS_NXDOMAIN` response for unregistered domain `fake.asad`.

```
PS C:\projects\Sentra_DNS> python src/client.py --resolver-ip 26.135.109.124 --domain short.neduet.local
[CLIENT] Sending MYDNS/1.0 query for 'short.neduet.local' to 26.135.109.124:53000...
======================================================================
MYDNS/1.0 Resolution Result
======================================================================
Queried Domain:   short.neduet.local
Resolver Host:    26.135.109.124:53000
Transaction ID:   0x3d86 (15750)
Response Status:  SERVER_FAILURE (0x02)
Round-Trip Time:  26.52 ms
Resolved IPv4:    None (0.0.0.0)
Time-To-Live:     0 seconds
----------------------------------------------------------------------
[SERVER FAILURE] Resolver was unable to reach authoritative upstream server.
======================================================================
PS C:\projects\Sentra_DNS>
```

(a). Host 1 Client Output: Graceful `STATUS_SERVER_FAILURE` Recovery When Host 3 Is Offline (RTT = 26.52 ms)

```
PS D:\Projects\SentraDNS> python src/resolver.py --port 53000 --upstream-host 26.1.9.90 --upstream-port 53053
[RESOLVER] listening on ('0.0.0.0', 53000), upstream 26.1.9.90:53053
[RESOLVER] ('26.225.66.37', 63249) MISS 25.52ms
[RESOLVER] ('26.225.66.37', 57554) MISS 23.86ms
[RESOLVER] ('26.225.66.37', 57555) HIT 0.08ms
[RESOLVER] ('26.225.66.37', 53657) SERVER_FAILURE 24.37ms
```

(b). Host 2 Resolver Log: Upstream Timeout Detection and Immediate Fail-Fast Relay (24.37 ms)

```
PS C:\projects\Sentra_DNS> python src/client.py --resolver-ip 26.135.109.124 --domain fake.asad
[CLIENT] Sending MYDNS/1.0 query for 'fake.asad' to 26.135.109.124:53000...
======================================================================
MYDNS/1.0 Resolution Result
======================================================================
Queried Domain:   fake.asad
Resolver Host:    26.135.109.124:53000
Transaction ID:   0x719f (29087)
Response Status:  NXDOMAIN (0x01)
Round-Trip Time:  26.77 ms
Resolved IPv4:    None (0.0.0.0)
Time-To-Live:     0 seconds
----------------------------------------------------------------------
[NXDOMAIN] Domain 'fake.asad' does not exist in authoritative zone database.
======================================================================
```

(c). Host 1 Client Output: Authoritative `STATUS_NXDOMAIN` Handling for Unregistered Domain (`fake.asad`)

Figure 8. Empirical Fault Injection and Negative Resolution Demonstrations (R7)

Member Name	Roll No.	Implementation & Report Contributions	Signature
Asadullah Zafar	CS-24082	Designed `MYDNS/1.0` binary wire contract (`!HBBBxH`); engineered `src/protocol.py` (pack/unpack SerDes engine), `src/client.py` (Stop-and-Wait ARQ, microsecond RTT profiler), and `tests/benchmark_runner.py`; authored Sections 1, 2, 6, 7, 9, and 10.	
Atta-Ur-Rehman	CS-24092	Engineered `src/resolver.py` (thread-safe `DNSCache`, 32-worker pool, upstream timeout tuning); conducted Wireshark capture analysis; authored Sections 3, 5, and 8.	
Abdur Rehman	CS-24098	Engineered `src/authoritative_server.py` and `src/zone.json` (zone lookup engine and strict wire-length enforcement); authored Section 4.	

Table 6. Group Member Contributions and Sign-Off

11. Individual Contributions

References

[1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[2] P. Mockapetris, "Domain Names — Implementation and Specification," IETF RFC 1035, Nov. 1987.

A. Declaration of AI Assistance

In accordance with Section 10 of the CS-327 CEP instructions, we declare the use of AI tools (Google Gemini and Anthropic Claude) during development:

- **Code Auditing & Edge-Case Verification:** Used to audit socket timeout ordering (Host 1 1.5 s vs. Host 2 1.0 s), trailing-byte validation, and unit test coverage against our protocol specification.
- **Report Formatting:** Used to structure the Overleaf LaTeX report template; all protocol designs, socket implementations, three-host deployments, packet captures, and empirical benchmarks were executed and verified directly by the group members.